

$\mathrm{T}_\mathrm{E}X$ ¹ の導入

1. $\mathrm{T}_\mathrm{E}X$

1.1. $\mathrm{T}_\mathrm{E}X$ のインストール. $\mathrm{T}_\mathrm{E}X$ とは:

- $\mathrm{T}_\mathrm{E}X$: フリーで使える数式を含む文書作成言語 (compiler).
D. Knuth により開発された.
- $\mathrm{L}_\mathrm{A}\mathrm{T}_\mathrm{E}X$: L. Lamport が $\mathrm{T}_\mathrm{E}X$ を使いやすくしたマクロパッケージ.
数式・相互参照・表・作図・プレゼン・hyperlink などが使える
- 高校の先生・高校生でも使ってる人がいるらしい.

英語表示の Mac の場合のインストールパッケージ:

- MacTeX: mac 用 installing package (以下を全て含んでいる)
- TeXLive: $\mathrm{T}_\mathrm{E}X$ の本体, mac だと /usr/local/texlive にインストールされる.
- TeXShop: $\mathrm{T}_\mathrm{E}X$ の mac 用の editor (開発環境で, VSCode などでも使用可能)
- beamer: プレゼンのための $\mathrm{L}_\mathrm{A}\mathrm{T}_\mathrm{E}X$ の documentclass
- METAPOST: $\mathrm{T}_\mathrm{E}X$ の描画ソフト (font 作成ソフト METAFONT([5]) から派生した描画言語)

演習. $\mathrm{L}_\mathrm{A}\mathrm{T}_\mathrm{E}X$ のインストール (Mac の英語表示のみ) :

- (1) google で MacTeX を検索
- (2) MacTeX をダウンロード
- (3) MacTeX.pkg をダブルクリックし, インストーラーの指示に従いインストール.
- (4) texshop で 設定が必要かも知れない. 詳細は [texwiki](#) を参照

texlive のファイル群は /usr/local/texlive という不可視フォルダーにインストールされている. コマンドラインユーザーインターフェイスでは簡単にアクセスできるが, (Mac の) グラフィカルユーザーインターフェイスでは Finder のメニューバーの go のメニューの Go to Folder を選ぶとフォルダーの選択窓が開くので /usr/local/texlive と打ち込むと移動できる.

Windows などの場合には [3] などを参考にしてインストールして下さい.

1.2. $\mathrm{T}_\mathrm{E}X$ のタイプセット. 次のソースファイル

```
\documentclass[11pt]{article}
\usepackage{graphicx, xcolor}
\usepackage{latexsym}
\usepackage{amsmath}
```

¹ $\mathrm{T}_\mathrm{E}X$ は American Mathematical Society の登録商標だが, このように明記する必要は無いらしい.

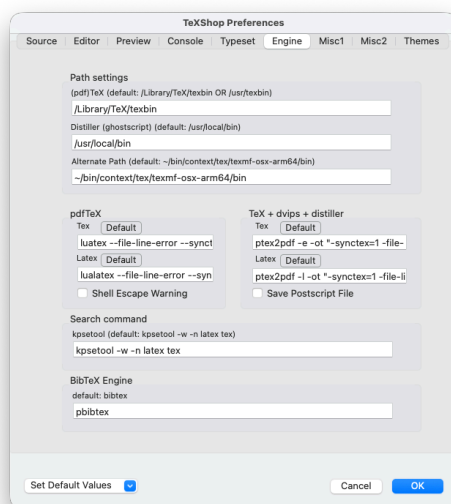


図 1. texshop の設定

```

\usepackage{amssymb}
\usepackage{amsthm}
\renewcommand{\figurename}{図}
\renewcommand{\proofname}{証明}
\renewcommand{\tablename}{表}
\DeclareMathOperator{\const}{const}
\title{Hello world \TeX !}
\begin{document}
\maketitle

```

因数分解 $x^3-1=(x-1)(x^2+x+1)$.

不定積分

```

\[
\int \sin x, dx = -\cos x, \quad
\int \frac{1}{x} dx = \log |x|

```

行列

```

\begin{equation}\label{eqMat}
B
=
\begin{pmatrix}
1&0&0\\
1&1&0\\
1&1&1
\end{pmatrix}
\end{equation}

```

`\eqref{eqMat}`のように引用できる. 標準でない記号例えば`\const$`も使うことが出来る (`\verb/$const$ /` とすると`$const$`になる).

`\textcolor{red}`{色も付けられる}. 表も作れる:

```
\begin{table}[thb]
  \centering
  \begin{tabular}{l|llll}\noalign{\hrule height 1pt}
    & $\varepsilon$ & $\alpha$ & $\beta$ & $\gamma$ \\
    \hline
    & $\varepsilon$ & $\varepsilon$ & $\alpha$ & $\beta$ & $\gamma$ \\
    & $\alpha$ & $\alpha$ & $\varepsilon$ & $\gamma$ & $\beta$ \\
    & $\beta$ & $\beta$ & $\gamma$ & $\varepsilon$ & $\alpha$ \\
    & $\gamma$ & $\gamma$ & $\beta$ & $\alpha$ & $\varepsilon$ \\
  \noalign{\hrule height 1pt}
  \end{tabular}\caption{Kline group}
\end{table}
```

`\noindent`

以下のように文献表も付けられる (`\cite{K}`).

```
\begin{thebibliography}{9}
\bibitem{K} Donald E. Knuth, \textit{The TeX}, book,
Addison-Wesley, 1984.
\end{thebibliography}
\end{document}
```

を `helloWorld.tex`² として保存し (読み込みたい画像とかのファイルがあればそのディレクトリに置き), `helloWorld.tex` を `TeXShop.app` で開き, `typeset` をクリックするか, `cmd T` とタイプするとコンパイルできる (図 2).

演習. 上のソースファイルをコンパイルして pdf を表示してください.

```
\begin{document}
```

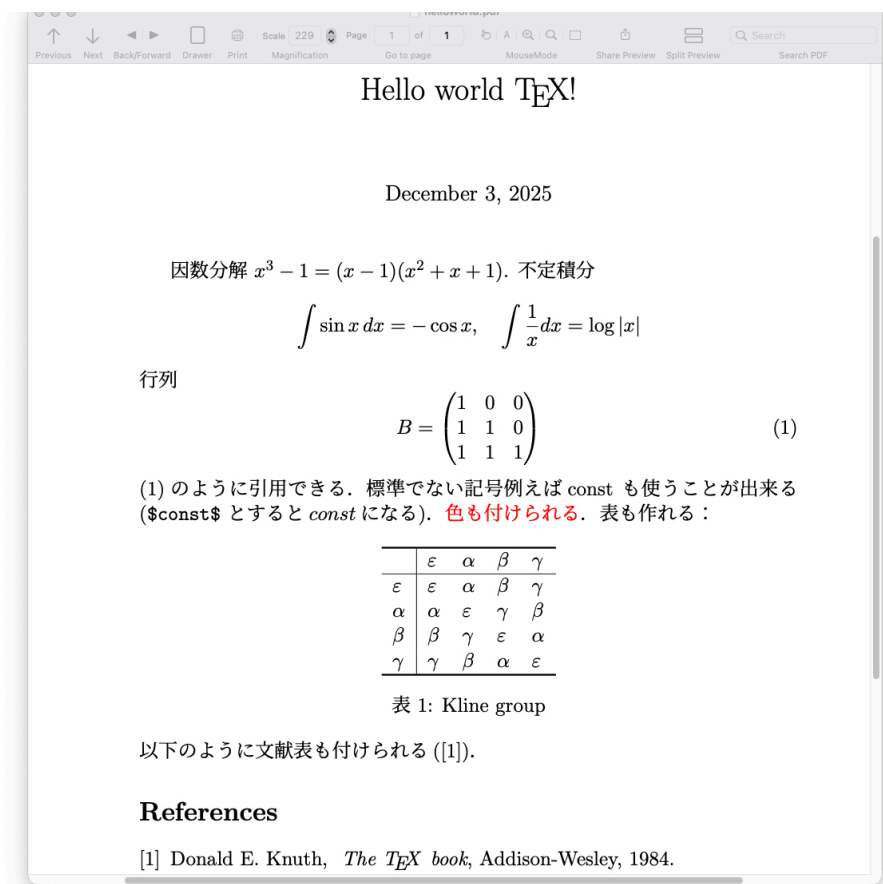
の前を `preamble` (序文) という. 取りあえず `preamble` はコピーして使い, 簡単な数式を [1] などを調べながら書いてみることをお奨めする. ひとことコメントすると

```
\DeclareMathOperator{\const}{const}
```

が標準にない記号 `const` を定義しているところである. あと, `TeX` の機能と云うより `TeXShop` (`Editor`) の機能だが,

- `cmd` キーを押しながらクリックすると対応する pdf の場所とソースファイルの場所を行き来することができる.

²どんなコンピュータ言語でも最初を書くプログラムの名前を `helloWorld` のようなものにするのがプログラマーの伝統である. また, どうでも良い名前の場合 `hoge` とか `fuga` のような名前を使うのも伝統である.

図 2. Hello world T_EX!

- コンパイルしてエラーが出て止まったとき, console の上部の Goto Error をクリックすると, ソースファイルの該当部分に飛ぶことができる.
- ソースファイルで Esc を押すと補間機能を使うことが出来る. 例えば `\begin{eq` まで打って Esc を押すと

```
\begin{equation}
.
\end{equation}
.
```

と補完される.

また, `emath` という大熊一弘氏の開発した日本の小中高数学のプリ

ント作成用の `latax` の style file があり, 教科書の数式に近い出力を得ることが出来る. [ここから](#) ファイルを取り寄せ, `/usr/local/texlive/texmf-local/tex/latex/local/` にコピーし, terminal から

```
% sudo texhash
```

として path を通す. ただし `%` はプロンプトで, タイプしていけない. preamble に

`\usepackage{emath}`

を追加すると使えるようになる．詳しくは `emath` のマニュアルを参照．

1.3. $\text{\TeX}$ の図式作成ソフト．

$$\begin{array}{ccc} S^2(1) & \longrightarrow & \mathbf{R}^3 \setminus \{0\} \\ \mu' \downarrow & & \downarrow \mu \\ S^2(1)/\sim & \xrightarrow{\pi} & \mathbf{P}^2 \end{array}$$

のような図式を描く，`texlive` に含まれるソフトの一つが `XY-pic` である．まず，

```
\input xy
\xyoption{all}
```

を `preamble` に付け加え，本文中に

```
\[
  \xymatrix{
    S^2(1) \ar[d]_{\mu'} \ar[r]
    & \mathbf{R}^3 \setminus \{\mathbf{0}\} \ar[d]^{\mu} \\
    S^2(1)/\sim \ar[r]_{\pi} & \mathbf{P}^2
  }
\]
```

を入れると，上の図式を描くことができる．`\xymatrix{...}` の中でマトリックスを描く要領で成分の数式の記号を並べている．`&` は列の分割を表し，`\\` は改行である．`\ar[d]` は下への矢印，`\ar[r]` は右への矢印を描く指令で，それに続く `_{\mu'}` は下付き，`^{\mu}` は上付きの矢印の修飾を描く命令である．

1.4. $\text{\TeX}$ の描画ソフト．`METAPOST` のコンパイルは，ソースファイル `figure.mp` を `texshop` で開き，ファイルのウィンドウの上部にあるファイルタイプのセクターを `LaTeX` から `MetaPost` に変更し，タイプセットする (図 3) と，`figure.1`，`figure.2` のような `postscript` ファイルが生成される．あるいは，ファイルのあるディレクトリで `terminal` から

```
% mpost figure.mp
```

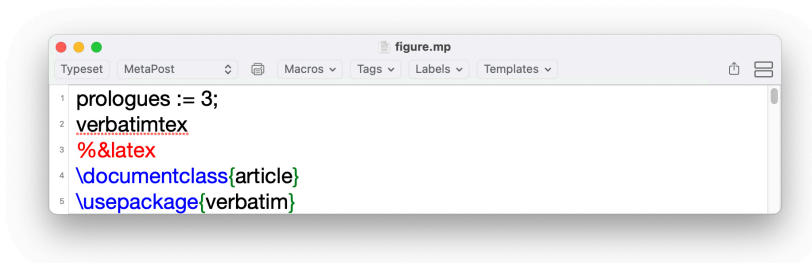


図 3. `METAPOST` のコンパイル

でコンパイル出来る．

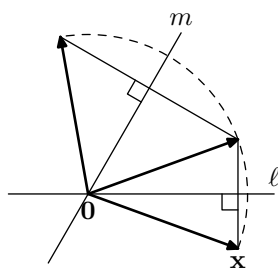


図 4. META-
POST の描画
例

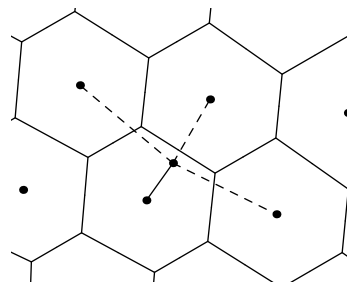


図 5. META-
POST の描画
例

のような絵を描く METAPOST のソースファイルの例を次にあげる:

```

prologues := 3;
verbatimtex
%&latex
\documentclass{article}
\usepackage{verbatim}
\usepackage{latexsym}
\usepackage{amssymb}
\usepackage{amsmath}
\begin{document}
etex;

numeric a, b;
a=10pt;
b=6pt;

beginfig(1);
  z0=(5a,3a);
  z1=z0-(3a,0);
  z2=z0+(7a,0);
  draw z1--z2; %line \ell
  label.top(btex $\ell$ etex,z2);
  z5=z0+(7a*cosd(60), 7a*sind(60));
  z6=z0-(3a*cosd(60), 3a*sind(60));
  draw z5--z6;%line m
  label.top(btex $m$ etex,z5);

  for j=-2 upto 10:
    z[100+j]=z0+6a*(cosd(10j),sind(10j));
  endfor;

```

```

draw
  for i=-2 upto 9:
    z[100+i]..
  endfor
  z110 dashed evenly;

z150=whatever[z0,z2]=whatever[z98,z102];
z160=whatever[z0,z5]=whatever[z102,z110];
z151=z150-(b,0);
z152=z151-(0,b);
z153=z152+(b,0);
z161=z160-(b*cosd(60),b*sind(60));
z162=z161+(b*cosd(150),b*sind(150));
z163=z162+(b*cosd(60),b*sind(60));

draw z98--z102--z110;
draw z151--z152--z153;
draw z161--z162--z163;

label.bot(btex $\mathbf{0}$ etex,z0);
label.bot(btex $\mathbf{x}$ etex,z98);

pickup pencircle scaled 1pt;
drawarrow z0--z98;
drawarrow z0--z102;
drawarrow z0--z110;
endfig;

beginfig(2);
def voro(expr p, q, r) =
  begingroup
    save a, b, c, u, v, w, e, f, g, h;
    pair a, b, c, u, v, w;
    numeric e, f, g, h;
    a = 0.5[q,r];
    b = 0.5[r,p];
    c = 0.5[p,q];
    (e,f)= r-q ;
    (g,h) = p-r;
    u = (f, -e);
    v = (h, -g);
    w = whatever[a + u, a - u] = whatever[b + v, b - v] ;
    draw a--w--b;
    draw w--c;
  endgroup

```

```

enddef;

z0=(-5a,-5a);
for i=1 upto 6:
  for j=1 upto 6:
    z[10i+j]= z0 + (5i*a, 0) +5j*(a/2, sqrt(3)*a/2)
      +(a*cosd(30(i+j)+i*j/2), a*sind(30(i+j) + i*j/2) );
  endfor;
endfor;

z100 = z32+ (a,1.4a);
dotlabel(btex etex, z100);
draw z100--z32;
draw z100--z33 dashed evenly;
draw z100--z23 dashed evenly;
draw z100--z42 dashed evenly;

for i=1 upto 6:
  for j=1 upto 4:
    dotlabel(btex etex, z[10i+j]);
  endfor;
endfor;
for i=1 upto 5:
  for j=1 upto 3:
    voro(z[10i+j], z[10(i+1)+j], z[10i+j+1]);
    voro(z[10i+j+1], z[10(i+1)+j+1], z[10(i+1)+j]);
  endfor;
endfor;
endfig;

verbatimtex
\end{document}
etex;
end

```

を figure.mp という名前で保存し、コンパイルすると figure.1, figure.2 というファイルが作られる。METAPOST のプログラミング言語としての仕様 (詳しくは [2]):

- beginfig(1); から endfig; まだが figure.1 の描画内容である。figure.2 も同様。これ以外の最初の部分と最初の部分はコピペすることをお奨めする。
- 文末には ; が必要。% の後がコメント文
- 点を表すのに、z10 のような z に数字を並べた表記を使う。インデックスを変数として扱うときは z[i] のような記法を使う。
- 点 z1 から z2 への直線を書くのは

```
draw z1--z2;
```

点 z_1 から z_2 への矢印を書くのは

```
drawarrow z1--z2;
```

z_1, z_2, z_3 を補間するベジェ曲線は

```
draw z1..z2..z3;
```

- METAPOST 特有の機能として

```
z150=whatever[z0,z2]=whatever[z98,z102];
```

のように直線と直線の交点を求めることが出来る。様々な初等関数も用意されている。関数は `sind(10)` のように呼び出される。

- `for` 文で点を定義したり、曲線を描くことも出来る。
- 図への数式や文字のラベルでは $\text{\LaTeX}$ と同じものを使える。例えば

```
label.bot(btex $\mathbf{0}$ etex,z0);
```

は点 z_0 の真下の部分に $\mathbf{0}$ (`$\mathbf{0}$`) を書くことを命令するコマンドである。

- `beginfig(2)` に関数の使い方の例をあげたのでぜひ読み解いてほしい。この例では `def` から `enddef` の中身 `volo` が三つの入力変数をもつ関数である。= の後がその定義である。定義の中で使う変数をまず保存して、その後変数の宣言を書く。次に二点の中点をもとめ、二点の作る辺に直交するベクトルをもとめその中点に直交する線分をもとめている。もう一つの二点についても同様に中点に直交する線分を求めそれらの交点として三点に外接する円の中心をもとめている。そこから各中点まで線分を3つ書いている。

あとは正六角形格子の点を少し振動させ、それらの点についてのボロノイ分割を `volo()` を使って書いている。

- ここではこれ以上述べないがもちろん `if` 文も使える。

演習. 上の METAPOST のソースファイルを `figure.mp` という名前で保存し、コンパイルし、上で作った `tex` のファイルに取込み `pdf` に表示してください。

`TikZ` と言う `style file` もあり、それを使うと直接 $\text{\LaTeX}$ の中で METAPOST に似たコマンドで描画できる。また `metafun` という METAPOST を現代風に拡張した `style file` もある。

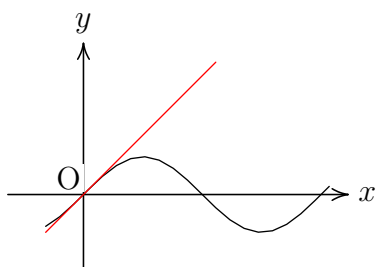
`MePoTeX` という (ここから入手可能) METAPOST を拡張する `style file` もあり、(`emath` と同じようにインストールし `path` を通せば) $\text{\LaTeX}$ の中で使える。 `preamble` に

```
\usepackage{MePoTeX}
```

を追加し、本文に

```
\begin{MPpic}<5mm>(7|2,4|2)
\mptXaxis[l]<1mm,0mm>{\$x\$}
\mptYaxis[b]<0mm,1mm>{\$y\$}
\sendMP{
  xdraw() kansu(sin(t))
  (-1,6.5,20);
  xdraw(red) kansu(t)
  (-1,3.5,20);
}
\mptLabel{origin}[tr]
<0mm,4mm>{0}
\end{MPpic}
```

を挿入すると



のような図が描ける.

レポート. 自分のセミナーの内容を説明する $ETEX$ のファイルを作成し、コンパイルせよ (目標は下の作業を含めて *pdf* で 5~10 ページほど). それに表・図式・参考文献を付け加えよ. *METAPOST* で図を描き, それをその説明の $ETEX$ の *pdf* の中に表示せよ.

以下の参考文献も参照

REFERENCES

- [1] TeXShop のメニューバーの Help メニューにある説明文書と動画
- [2] /usr/local/texlive/2024/texmf-dist/doc/latex 中のドキュメントファイル: User's guide for amsmath package, Using the amsthm package, XY-pic User's Guide, XY-pic Reference Manual, The METAPOST system, A users manual for METAPOST など
- [3] 奥村晴彦, 黒木裕介, [改訂第 9 版] $ETEX$ 美文書作成入門, 技術評論社, 2023.
- [4] *The $ETEX$ Companions Third Revised Boxed Set*, Addison-Wesley, 2007.
- [5] Donald E. Knuth, *The METAFONTbook*, Addison-Wesley, 1986.

2. 補足: MARKDOWN

T_EXとは直接関係はないが、最近よく使われるMarkdown言語についても紹介したい。これはWeb pageを編集するための言語html(=hyper text markup language)を簡易的に表すために、つまり、markdownを目指して開発された言語である。VSCodeでIntroToMarkdown.mdというファイルを作り、以下のソースをコピーし、cmd+shift+Vとタイプすると（またはウインドウの右上にあるノートに虫眼鏡のアイコンをクリックすると）図6, 7のようなhtmlのプレビュー画面が現れる。

```
<!-- Introduction to markdown created by snow00two -->
```

```
# Introduction to Markdown
```

```
## Markdown
```

```
- list1
- list2
  - sublist1
  - sublist2
- list3
```

```
1. list1
2. list2
3. list3
```

```
Separator:
```

```
---
```

```
*italic* , **bold** , ***italic bold***
```

```
> quotation
>> second quotation
```

```
Programming Language 'JavaScript' list:
```

```
```javascript
function hoge () {
 console.log("problem");
}
```
```

```
Table:
```

```
| Data | A | B |
```

```
| ---: |: ---: |: ---: |
| 0 | 1 | 2 |
```

Mathematics:

$\sqrt{2}$, π , $\frac{1}{x+1}$

Hyperlink to: [amazon](https://www.amazon.co.jp/amazonprime)

Image file: ![image file](dodecahedron.jpg)

文法についてはこのファイルを見れば直ぐに分かるだろう。最近の技術系ホームページ (例えば GitHub) は Markdown で作られていることが多い。また、生成 AI のファインチューニングという訓練で Markdown が使われていることはよく知られていて、ChatGPT へのプロンプトの入力も Markdown を使うことが推薦されている ([1], [2])。最後に markdown から L^AT_EX を作ることも出来る (とは言っても数式等は L^AT_EX で作らなければならない)。

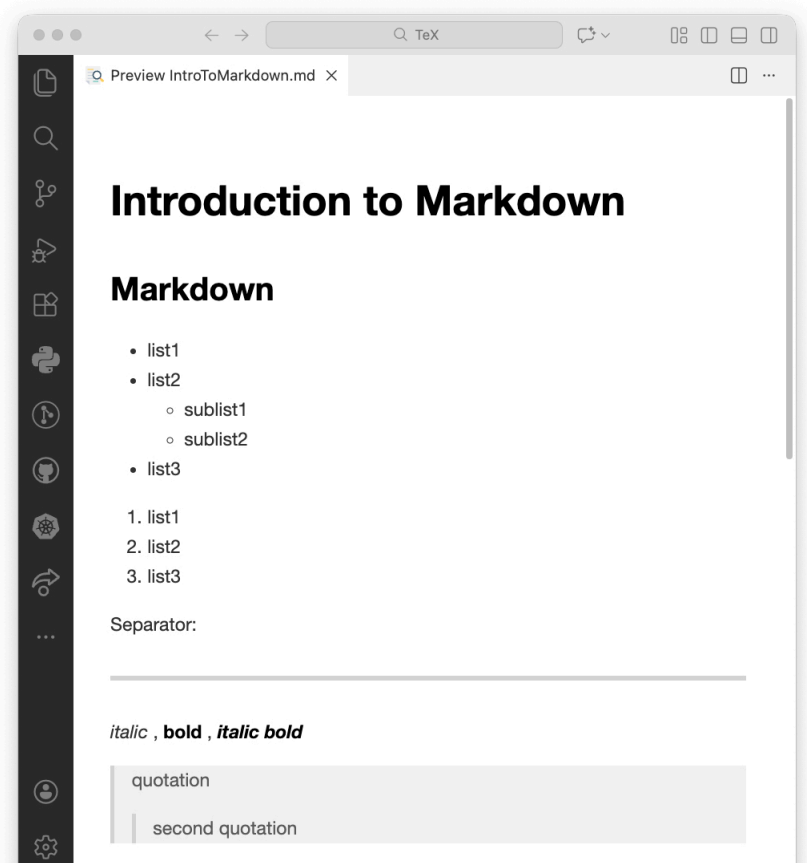


図 6. VSCode での IntroToMarkdown.md のプレビュー 1

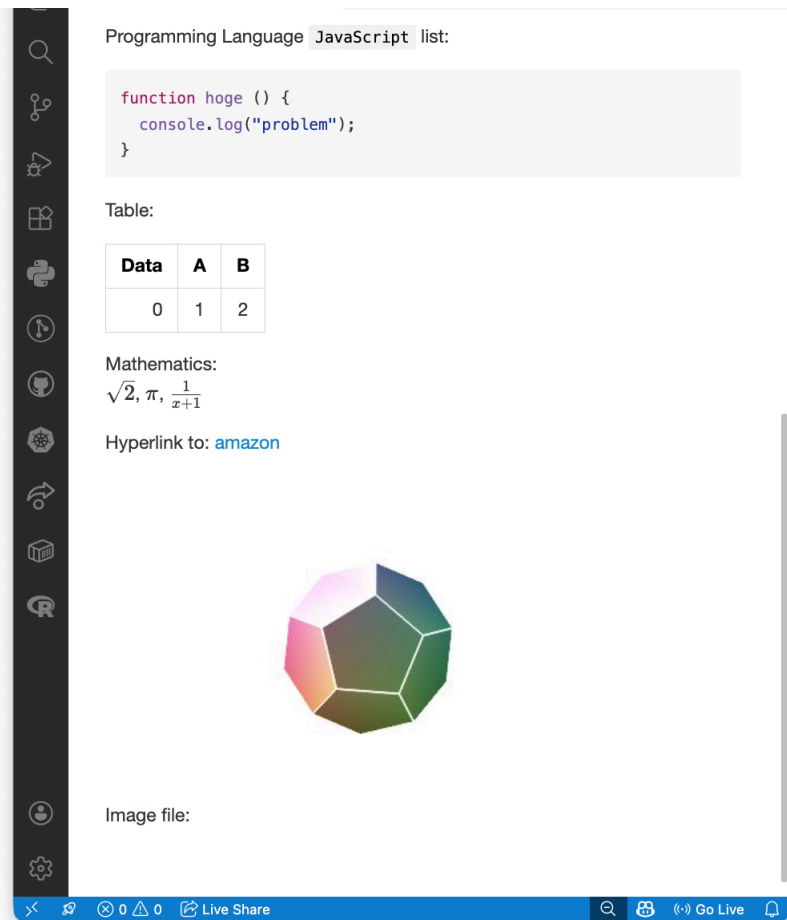


図 7. VSCode での IntroToMarkdown.md のプレビュー 2

- 演習.
- (1) VSCode で上のソースファイルを参考にして、1〜2 ページ程度で markdown を説明する markdown の文書を作ってプレビューして下さい。
 - (2) そのソースファイルを、生成 AI を使って latex のソースファイルに変換してください (その中でコマンドの実行を頼まれるかも知れません)。
 - (3) その過程で生成 AI が何をしたのか分析してください。
 - (4) その分析に基づき、自分一人でもう一度その latex の書類を markdown に変換して下さい。

REFERENCES

- [1] Berryman, Zigler, LLM のプロンプトエンジニアリング, オライリージャパン, 2025.
- [2] 平田 賀一, IT エンジニアのための Markdown 実践入門 —— 生成 AI 時代の”伝わる”書き方, 技術評論社, 2026.